

7 带容量的指派与背包：LP 不再自带整数解

第 5、6 章的问题都有一个共同点：约束矩阵全单模，LP 直接给整数解。这一章给车加上载重上限，第一次碰到例外。约束里出现了 0 和 1 以外的系数，LP 开始给出分数解，我们回到第 4 章的分支定界和割平面，看它们怎样在一个具体的问题上工作。

学完本章，你能

1. 写出 0-1 背包的 LP 上界（Dantzig 上界），说明它为什么至多一个分数变量，以及为什么上界向下取整仍然不是答案。
2. 手写带 LP 上界的分支定界，读懂一棵搜索树：分支、剪枝、更新最好解。
3. 把“车有载重上限”的派车写成广义指派（GAP），说明载重约束怎样破坏整数性，以及载重大到不起作用时整数性怎样回来。
4. 从 LP 解里分离覆盖割，看割怎样把根节点的下界拉近整数最优、缩小搜索树；用装载（装箱）的例子理解启发式和最优的差距。

本章信息

用时	阅读与交互 80-110 分钟，Lab 3 小时左右
前置	第 4 章（LP 松弛、分支定界、割平面）、第 5、6 章（全单模、指派、网络流）。会读 Python 代码。
对拍	SciPy 1.17.1 · HiGHS 1.12.0；本章的算法由页面内置代码实现，不依赖求解器。
配套	Lab 代码包：5 个练习 + 1 个挑战，含手写的背包分支定界、覆盖割循环，pytest 自动判分

PART 1
数学规划基础

PART 2
组合优化

PART 3
调度理论

PART 4
路径与波次

PART 5
大规模方法

PART 6
启发式

PART 7
多智能体时空规划

PART 8
动态与不确定

第二部分 组合优化：指派问题与匈牙利算法 → 最短路与网络流 → 带容量的指派与背包（本章） → 旅行商与车辆路径入门。

7.1 背包与 LP 上界

先看最简单的“容量”问题。一辆车这一趟最多载 22 个重量单位，有 8 个托盘等着上车，每个托盘有一个价值（比如紧急程度）和一个重量。装哪几个，总价值最大？

这是 0-1 背包。设 x_j 为 1 表示装第 j 个托盘：

$$\text{背包 } \max \sum_j v_j x_j, \quad \sum_j w_j x_j \leq Q, \quad x_j \in \{0, 1\} \quad \text{只有一行约束，但系数是 } w_j$$

只有一行约束，为什么难？因为系数 w_j 是任意的整数，不再是第 5、6 章的“每列一个 +1 一个 -1”。约束矩阵没有全单模的结构，LP 松弛的顶点不必是整数。

但这个 LP 好解得出奇。把托盘按**价值/重量**从大到小排好，依次装，装到某一个装不下为止；再把它装进去一部分，正好装满。这就是 LP 的最优解（Dantzig, 1957），它的值叫 **Dantzig 上界**。它有两个性质：

- 1 **至多一个分数变量**。只有一行约束，LP 的顶点至多有一个变量落在 0 和 1 之间，其余都在边界上。这个分数变量就是“装到一半”的那个托盘。
- 2 **是整数最优的上界，但不一定紧**。整数解也满足 LP 的约束，所以 LP 值 $\geq$ 整数最优。价值是整数时，上界还可以向下取整，但取整之后仍然可能比最优大。

整数最优可以用**动态规划**精确求： $dp[i][c]$ 表示前 i 个托盘、载重 c 时的最大价值，一个托盘要么装要么不装， $O(nQ)$ 时间。它是**伪多项式**的：载重每多一位数字，时间乘以 10。载重是几十、几百的托盘数时很好用；载重是毫米或克的时候就不行了。

先预测 背包的载重是 22。LP 的最优解是：价值/重量最高的四个托盘装满（重量 20，价值 104），再装第五个托盘（T6，价值 10，重量 7）的 $2/7$ ，上界 106.86。整数最优是多少？

106

105

104

低于 100

预测正确。 整数最优是 104：就是那四个托盘，把分数的 T6 舍掉。上界向下取整（106）比它大，说明取整不能当答案。载重换成 26 时，整数最优就不是舍入得到的了。下面的图已经解锁，拖动载重看。

载重 $Q = 22$

LP 上界

106.86

装到装不下为止，再装一部分

整数最优 (DP)

104

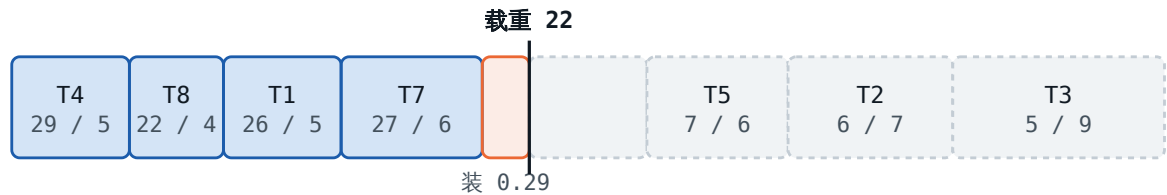
重量 20 / 22

差

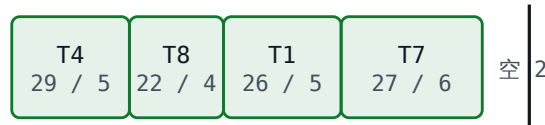
2.86

上界向下取整 = 106

LP 解



整数最优



■ LP 装满的托盘 ■ 被截断的那个 (LP 只装一部分) ■ 装不下 ■ 整数最优

按价值/重量从大到小装，装到 **T6** 时装不下，LP 只装了它的 **0.29**。至多一个变量是分数，其余都是 0 或 1。

LP 上界 106.86，整数最优 104。整数最优要另外选：T4、T8、T1、T7。整数解装不了 T6 的一部分，剩下的空间要用别的物品凑。上界向下取整 (106) 也比最优大，说明取整不能直接当答案。

物品	价值	重量	价值/重量	LP	最优
T4	29	5	5.80	1	选
T8	22	4	5.50	1	选
T1	26	5	5.20	1	选
T7	27	6	4.50	1	选
T6	10	7	1.43	0.29	
T5	7	6	1.17	0	
T2	6	7	0.86	0	
T3	5	9	0.56	0	

图 7-1 背包的 LP 上界。托盘按价值/重量从大到小排开，方块的宽度是重量，里面写着“价值 / 重量”。上一行是 LP 的解：一直装到载重线，被线截断的那个只装一部分（橙色）。下一行是整数最优（动态规划）。拖动载重看两者的关系。

读图

默认载重 22：价值/重量最高的四个托盘 T4、T8、T1、T7 合计重量 20，价值 104。剩下的 2 个重量单位，LP 用来装 T6 的 $\frac{2}{7}$ ，多得 2.86，上界 106.86。整数最优就是这四个托盘，价值 104，恰好是“舍掉分数变量”的结果。

拖到 26：LP 上界 112.57：四个托盘装 20，再装 T6 的 $\frac{6}{7}$ 。整数最优 111，是把分数的 T6 换成整个的 T5（价值 7、重量 6），正好把载重 26 装满。单是舍掉分数变量只有 104，差了 7。**整数最优不一定是 LP 解的舍入**，这是背包难的地方。

拖到 20：四个托盘正好装满，LP 没有分数变量，上界等于整数最优。这时问题恰好是“容易”的，但只是因为数据凑巧。

7.2 背包的分支定界

第 4 章讲过分支定界的原理：用 LP 的值当上界，在分数变量上分支，把子问题的上界和手上最好的整数解比较，没有希望的分支直接剪掉。背包正好可以把它完整地写出来，因为每个节点的 LP 都是一次排序加一次扫描。

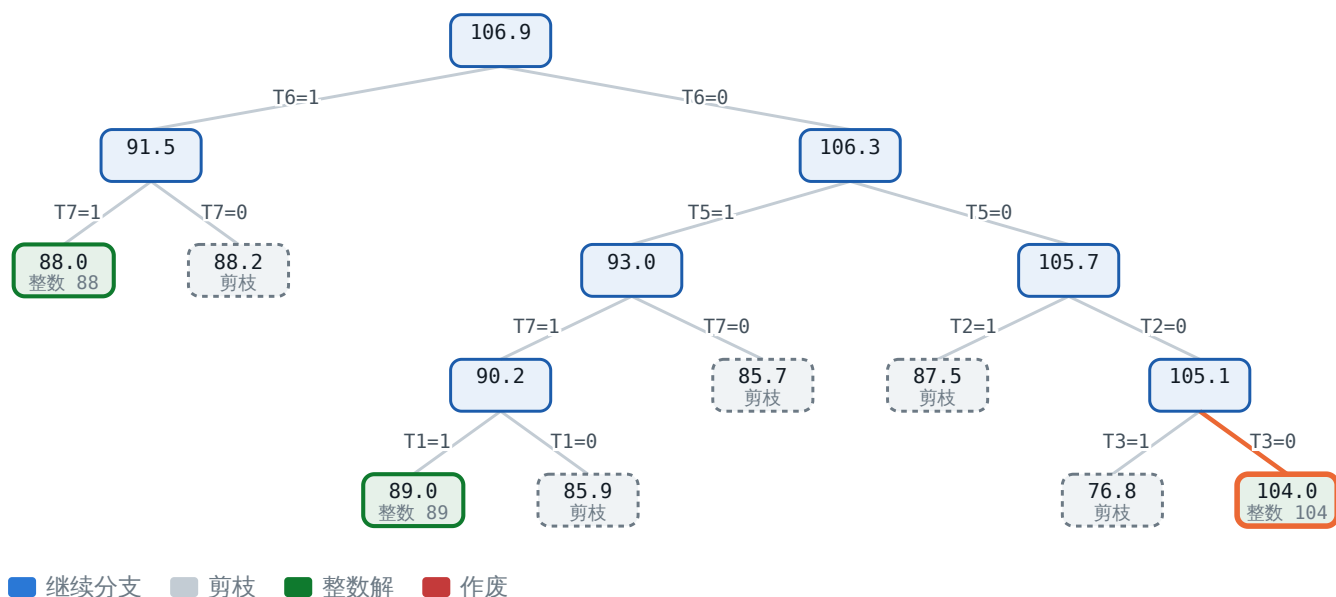
- 1 **节点**：一组已经决定的托盘（装或者不装）。它的**上界**是剩下的托盘在剩余载重下的 Dantzig 上界，加上已装托盘的价值。
- 2 **剪枝**：价值是整数，所以上界向下取整 $\leq$ 手上最好的整数解时，这一支不可能更好，不用再看。必装的托盘已经超载，这一支作废。
- 3 **整数解**：LP 解没有分数变量，这一支已经解完，更新最好的整数解。
- 4 **分支**：否则在 LP 解里那个分数变量上分成两支：装它、不装它。这里深度优先，先试“装”。

搜索树的大小取决于上界有多紧、最好的整数解找到得多早。8 个托盘完全枚举要看 511 个节点，加上上界剪枝之后只有十几个。

当前最好整数解：104

已创建 15 个节点

全部 15 个，完全枚举要 511 个



第 15 / 15 个节点。决定：T6 不选，T5 不选，T2 不选，T3 不选。上界 104.0。

找到整数解，价值 104，成为新的最好解。

图 7-2 同一个背包（载重 22）的分支定界树。每个方框里是这个节点的 LP 上界；连线上是分支的决定（ $T6=1$ 表示装 T6）。点“下一个节点”按深度优先的顺序逐个展开。

读图

根节点的上界是 106.86，分数变量是 T6，所以第一次分支是“装 T6”和“不装 T6”。深度优先先试装 T6。

“装 T6”一支的上界只有 91.5。LP 原本几乎没用 T6，硬把它装进去，损失很大。这一支很快找到整数解 88，然后剩下的节点因为上界 88.17 向下取整不超过 88 被剪掉。手上有了第一个整数解 88。

“不装 T6”一支的上界 106.33，很有希望，继续向下。接着试装 T5、不装 T5，一路下去，中间又找到 89，最后一个节点（不装 T2、不装 T3）的 LP 恰好是整数解，价值 104，与根节点的上界 106.86 相差 2.86。因为上界向下取整是 106，还高于 104，要把所有上界大于 104 的分支查完才敢确定 104 是最优，所以树里有十来个节点。

这与第 4 章的经验一致：下界靠早点找到好解，上界靠 LP 松弛紧。下一节的广义指派里，好解还比较容易靠启发式找到，难的是上界。7.4 节用覆盖割把 LP 上界收紧，就是从这一头减少节点。

7.3 广义指派：载重让整数性消失

第 5 章的指派有一条规定：每辆车至多做一个任务。现实里一辆车一趟可以做好几个，只要载得下。把“至多一个”换成“载重上限”，就得到**广义指派问题（GAP）**。

本章的例子：3 辆车 V1、V2、V3，载重分别是 15、16、17 个托盘；10 个任务，托盘数分别是 4、3、4、6、4、3、3、8、3、2，一共 40 个。每辆车做每个任务的行驶时间不同，目标是总行驶时间最小。每个任务恰好一辆车，每辆车分到的托盘数不能超过载重。

$$\text{GAP} \quad \min \sum_{ij} c_{ij}x_{ij}, \quad \sum_i x_{ij} = 1, \quad \sum_j w_j x_{ij} \leq Q_i, \quad x_{ij} \in \{0, 1\} \quad \text{每辆车的载重约束是一行背包}$$

对比第 5 章：任务约束（每个任务恰好一辆车）不变，车的约束从“至多一个任务”变成“托盘数之和不超过载重”。每辆车的这一行就是一个背包约束，系数是 w_j 。第 5 章的全单模证明用到“每列恰有两个 1”，现在每一列除了任务行的 1，还有载重行的 w_j ，证明不再成立。

这个问题是 NP 难的：它包含了装箱这样的问题，没有已知的多项式算法。LP 松弛仍然好解，但顶点可以是分数。一个粗略的计数：变量 mn 个，约束 $n + m$ 行，LP 的一个顶点至多有 $n + m$ 个非零变量；每个任务至少占一个，所以至多 m 个任务被劈给几辆车。车越少，分数越少，但只要有载重，就可能有。



图 7-3 广义指派（3 辆车 × 10 个任务）。格子里是行驶时间（秒），表头一行是托盘数，最右边是这辆车的载重使用情况（LP 解里加权的托盘数 / 载重）。拖动载重的加减量，看 LP 与整数最优的关系。LP 用页面内置的单纯形法求解，整数最优由回溯枚举得到。

读图

默认载重（加减 0）：LP 值 220，整数最优 234，差 14。LP 的解里只有 T3 被劈开：V1 和 V2 各负责一半。T3 有 4 个托盘，V1 的 15 个托盘已经被别的任务占满，装不下整个 T3，LP 就让 V2 顺手再装一半。整数解做不到，要重新安排好几个任务，总时间多 14 秒。

把载重调大：加 2 以后，LP 值 212，整数最优也是 212，两者相等。这时每个任务拿走最便宜的车，三辆车分到的托盘数是 16、18、6，都在载重之内，载重约束没有起作用，问题退化成“每个任务选最便宜的车”，整数性回来了。加 3、4 时 LP 值仍是 212，但求解器返回的顶点里 T9 仍是分数。原因是这一行有另一个整数的最优解，返回哪一个顶点取决于算法，整数性没有了保证。

把载重调小：减 1 时 LP 225，整数最优 234；减 2 时 LP 230，整数最优 239；减 3 时三辆车合计只能载 39 个托盘，装不下 40 个，无解。载重越紧，越靠近装箱问题，LP 和整数最优的差越不稳定。

整数性什么时候还在

载重不起作用（每辆车都装得下它最便宜的那些任务）时，GAP 就是没有容量的指派，LP 解是整数。载重一旦起作用，LP 就会把一个任务劈给几辆车。第 5 章的冲突对让 LP 出现“两个派法各一半”，这里是“一个任务分给两辆车各一半”，来源不同，症状类似。

7.4 覆盖割：把 LP 拉回来

LP 松弛的下界是 220，整数最优是 234，中间的 14 秒是 LP 的“虚假收益”。直接分支可以解出来，但节点会多。第 4 章讲过另一条路：加割平面，把 LP 的可行域收紧，同时不删掉任何整数解。

每辆车的载重约束是一个背包，背包有一类经典的割，叫**覆盖割**。

覆盖

任务集合 S 是车 i 的一个**覆盖**，如果 S 里所有任务的托盘数之和超过 Q_i 。极小覆盖：去掉其中任何一个，就装得下了。

覆盖割

覆盖 S 里的任务，车 i 至多装其中 $|S| - 1$ 个： $\sum_{j \in S} x_{ij} \leq |S| - 1$ 。

有效：全装进去就超载了，所以任何整数可行的派法都满足它，没有整数解被删掉。

分离

LP 解里，如果某个极小覆盖上的 x 之和超过 $|S| - 1$ ，这条割就被违反了，可以加进去，删掉当前这个分数解。

这是启发式的分离：按 x 从大到小取任务，取到超载为止，再去掉多余的任务使之极小。

做法就是**根节点上的割平面循环**：解 LP，对每辆车找违反的覆盖，加入，再解 LP，直到找不到为止。每加一轮，下界只增不减，且始终不超过整数最优（因为割是有效的）。

先预测 广义指派例子里，加载重之后 LP 值是 **220**，整数最优是 **234**。在根节点上反复“解 LP、找违反的覆盖割、加割”，下界最多能被推到多少？

约 222，割只能挤出一

点 约 226

233，几乎追平整数最优

超过 234，因为割更强

预测正确。 三轮之后下界是 233，页面找不到更多违反的覆盖割。不可能超过 234：割对每个整数可行解都有效，整数最优的解自己满足所有的割。下面的图已经解锁。

分支定界：不加割 55 个节点

先加 4 条割：23 个节点

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	载重
托盘数	4	3	4	6	4	3	3	8	3	2	
V1	49	38 1/5	32 3/5	6	29	45	31	57	38 2/5	44 2/5	15 / 15
V2	15	39	18 2/5	60	45	39	45	18	38 2/5	18 3/5	16 / 16
V3	59	39 4/5	44	42	59	13	19	52	45 1/5	56	9 / 17

第 3 轮的 LP：**233**，带着前 2 轮加的 4 条割。分数的任务：T2、T3、T9、T10。

再也找不到违反的覆盖割（按这里的分离规则）。下界停在 233，整数最优是 234，还差 1。剩下的这一点靠分支解决。

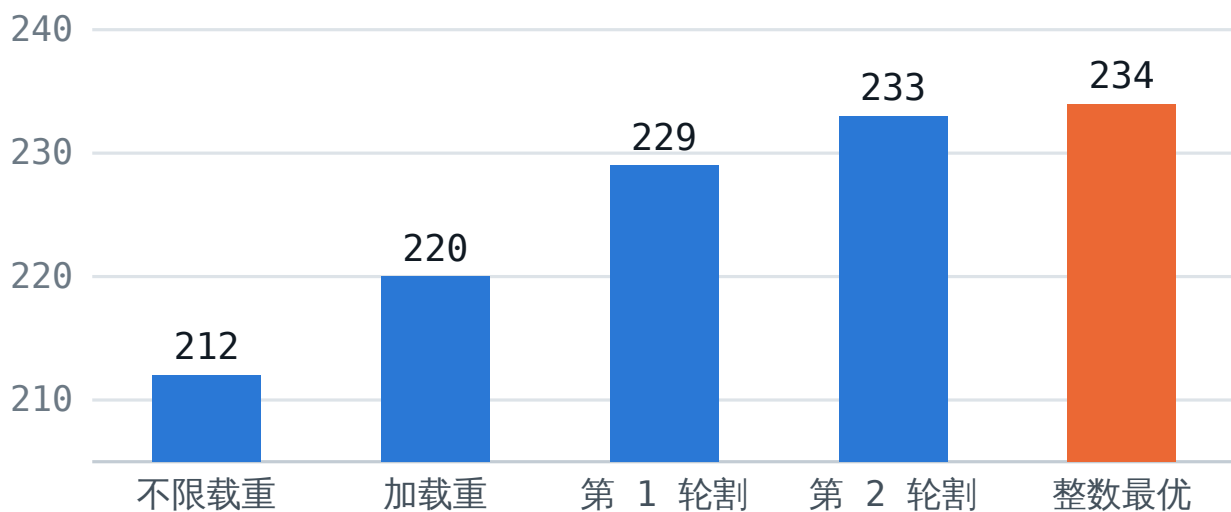


图 7-4 根节点的覆盖割循环（载重 15 / 16 / 17）。左：每一轮的 LP 解（橙色是分数）；右：下界的变化，最右边是整数最优 234。顶上的两个数字是分支定界的节点数：不加割，和先加这几条割再分支。

读图

第 1 轮只有载重约束，LP 220，T3 被劈给 V1 和 V2。页面分离出两条割：V1 上的 T3、T4、T5、T9（托盘数 $4 + 6 + 4 + 3 = 17 > 15$ ）至多选 3 个；V2 上的 T1、T3、T8、T10（ $4 + 4 + 8 + 2 = 18 > 16$ ）至多选 3 个。LP 解里这两组的和都是 3.5，被违反了。

第 2 轮加了两条割以后 LP 上升到 229。新的分数解又违反了两条覆盖：V1 上的 T2、T3、T4、T5，V2 上的 T1、T8、T9、T10。**第 3 轮** LP 233，页面再也找不到被违反的覆盖割。整数最优是 234，还剩 1 秒的差距，这一点留给分支。

树小了一半以上。不加割，分支定界要访问 55 个节点；把这 4 条割加进每个节点的 LP 之后，只要 23 个。割没有解决问题，但让每个节点的上界更紧，更早剪枝。求解器的日志里“根节点上加了多少割、下界从多少涨到多少”，就是这一步（第 4 章读日志那一节）。

拉格朗日松弛看同一件事

把载重约束乘以价格 $\lambda_i \geq 0$ 吸收进目标：每个任务独立地选“加价后”最便宜的车，是一个没有载重的问题，最优 λ 给出的下界正好等于 LP 值 220（Lab 挑战里验证）。原因是松弛后的子问题有整数性，所以拉格朗日追不过 LP。覆盖割能越过 220，是因为它用到了整数性（“装不下”）这一点。这个对比在第 19 章会再用到。

7.5 装载与启发式

广义指派里车已经固定，问的是“谁做哪个任务”。还有一种更基础的问法：**这批托盘最少要几个车次装完？**每个车次的容量一样，这是装箱问题（bin packing），也是装载计划最常见的形式。

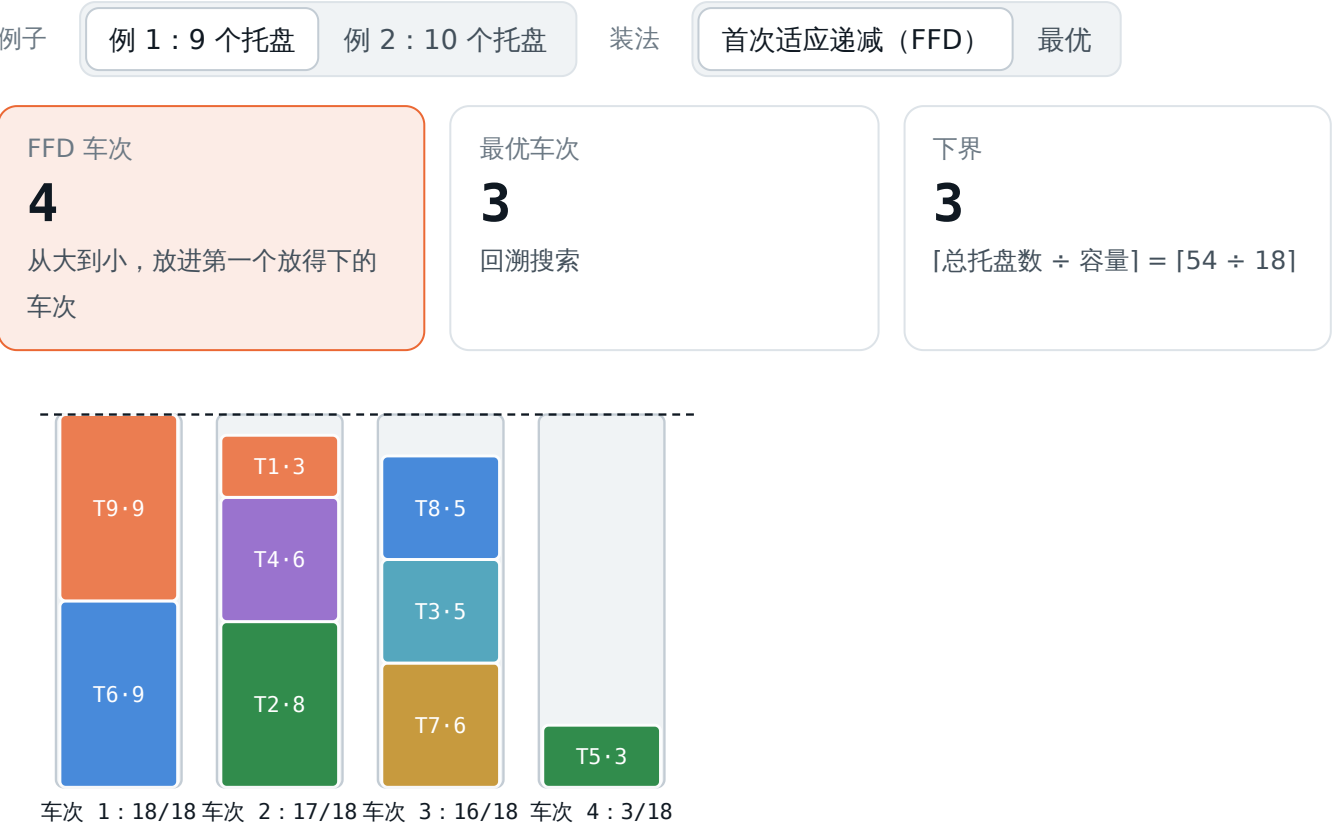
它是 NP 难的，但有一个简单的启发式：**首次适应递减**（FFD）。把托盘按重量从大到小排好，逐个放进第一个还放得下的车次，放不下就新开一个车次。它快，也好写。一个自然的问题是：它离最优有多远？

先给一个下界。所有托盘的总重量是 W ，每个车次至多装 Q ，所以至少要 $\lceil W / Q \rceil$ 个车次，这是装箱的 LP 松弛值（向上取整）。FFD 有已知的保证：车次数不超过 $11/9 \times \text{最优} + 6/9$ （Dósa，2007），而且这个系数是紧的。

先预测 9 个托盘的重量是 3、8、5、6、3、9、6、5、9，总重 54，每个车次容量 18。下界是 3 个车次。首次适应递减（从大到小，放进第一个放得下的车次）需要几个？

- 3 个
- 4 个
- 5 个
- 6 个

预测正确。 FFD 用 4 个：[9, 9]、[8, 6, 3]、[6, 5, 5]，最后一个 3 放不下任何一个车次。最优是 3 个：{9, 6, 3}、{9, 6, 3}、{8, 5, 5}，每个车次正好装满。下面的图已经解锁。



总共 54 个托盘，每个车次装 18，至少要 3 个车次。

FFD 用了 4 个车次，最优是 3。 FFD 先把大的放好，后面的小托盘补不满前面留下的缝，多开了一个车次。切换到“最优”看怎么拼。

最优解把每个车次**正好装满**：没有一点浪费，所以贪心很难恰好拼对。

图 7-5 装载。每一列是一个车次，方块的高度是托盘的重量（标注“托盘编号·重量”），虚线是容量。切换“首次适应递减”和“最优”，看车次数的差别。最优由回溯搜索得到。

读图

例 1：9 个托盘总重 54，容量 18，下界 3。FFD 先把两个 9 装满第一个车次，再把 8 和 6 放进第二个，接着 6、5、5 放进第三个，最后两个 3 一个塞进第二个车次的缝里，另一个再也放不下，只好开第四个车次。最优是 {9, 6, 3}、{9, 6, 3}、{8, 5, 5}，三个车次**每个都正好装满**。FFD 一开始把两个 9 放在一起，破坏了这种拼法。

例 2：10 个托盘，容量 10，FFD 用 5 个车次，最优是 4。

这两个例子说明启发式的价值和局限。FFD 几乎总是很好，在这样的小例子里最多差一个车次；但“总重恰好是容量的整数倍”这种紧的实例，正是它容易出错的地方。要知道离最优多远，就需要下界。最好的下界通常来自 LP 松弛或者更强的松弛，第 17 章的列生成对装箱给出的下界比这里的 $\lceil W/Q \rceil$ 更紧。

7.6 在项目里的位置

载重和容量在四向车与提升机的场景里到处都是：车的载重、提升机一次的载运量、波次对月台和发运车辆的容量。判断该用什么方法，先看规模和你要的答案。

几十个任务，要最优或者带证书的解

直接用 MIP 求解器建 GAP 模型。本章的覆盖割，现代求解器在根节点上会自动会加（还有提升过的版本）。你要做的是读日志：根节点下界涨了多少，gap 是多少（第 4 章）。

几百上千个任务，要快

用启发式：贪心（按“遗憾值”把任务派给最便宜的车）加局部搜索（把任务换到别的车上）。第 21 章会做。好解还能给求解器当热启动。

装载、下料、按车次分组

用“方案”建模：一个变量代表一种装法，约束只有“每个托盘恰好被装一次”。LP 松弛比“托盘 $\times$ 车次”的模型紧得多，代价是方案数是指数的，要靠列生成（第 17 章）。

不管选哪种，先把 LP 松弛跑一遍，把它和整数解的差记下来。差为 0 或者很小，说明载重约束不怎么起作用，简单的方法就够；差很大，说明约束是真正的瓶颈，值得花力气在割、分解或者更好的模型上。这条经验适用于本课程后面所有带容量的问题。

另外，第 5、6 章的“整数性”要留在脑子里：一个带容量的模型如果能拆成“若干个背包 + 一个指派”，那么指派那部分是容易的，背包那部分是难的，这个分界就是后面分解方法（第 17-19 章）

的入口。

7.7 小结与 Lab

- 1 0-1 背包的约束矩阵只有一行，系数是任意整数，没有全单模结构。它的 LP (Dantzig 上界) 按价值/重量装到装不下，再装一部分，至多一个分数变量。上界向下取整仍然不一定等于最优，整数最优也不一定是 LP 解的舍入。
- 2 动态规划是 $O(nQ)$ 的伪多项式算法，载重的数值大时不适用。
- 3 分支定界：用 Dantzig 上界当每个节点的上界，在分数变量上分支，上界向下取整不超过最好的整数解时剪枝。深度优先能早点得到整数解，剪枝更多。
- 4 广义指派 (GAP) 把第 5 章的“每车至多一个任务”换成载重上限。载重行的系数破坏了全单模，LP 会把一个任务劈给几辆车，至多 m 个任务。载重大到不起作用时，整数性回来。
- 5 覆盖割：装不下的任务集合，车至多装其中 $|S| - 1$ 个。它对所有整数可行解有效，同时能删掉当前的分数解。根节点上反复“解 LP、找违反的覆盖、加割”，下界只增不减，本章例子里从 220 涨到 233，整数最优是 234，分支定界的节点数从 55 降到 23。
- 6 把载重约束乘价格吸收进目标，得到拉格朗日下界，最好也只等于 LP 值（子问题有整数性），追不过覆盖割。
- 7 装载（装箱）是 NP 难的。FFD 快，车次数不超过 $11/9 \times \text{最优} + 6/9$ ，但在总重恰好是容量整数倍的实例上容易多开一个车次。下界 $\lceil W/Q \rceil$ 用来判断离最优有多远。

术语表

中文	English	在本章的含义
0-1 背包	0-1 knapsack	每个物品装或不装，总重量不超过载重，总价值最大
Dantzig 上界	Dantzig bound	背包 LP 松弛的最优值：按价值/重量装，最后一个装一部分
伪多项式	pseudo-polynomial	时间与数值（如载重）本身成正比，而不是与它的位数成正比
分支定界	branch and bound	在分数变量上分支，用 LP 上界剪枝
广义指派	generalized assignment (GAP)	任务分给车，车有载重上限，一辆车可做多个任务
覆盖、覆盖割	cover, cover inequality	总重量超过载重的任务集合 S ； $\sum_{j \in S} x_{ij} \leq S - 1$
分离	separation	在当前 LP 解里找一条被违反的有效不等式
有效不等式	valid inequality	所有整数可行解都满足的不等式
装箱、FFD	bin packing, first-fit decreasing	用最少的车次装完全部托盘；从大到小放进第一个放得下的车次
拉格朗日松弛	Lagrangian relaxation	把一类约束乘价格吸收进目标，得到下界 $L(\lambda)$

对照教材

- Wolsey , *Integer Programming* (第 2 版 , Wiley 2020) : 背包问题的动态规划与覆盖不等式、提升的一节，以及分支定界和割平面的章节。
- Kellerer、Pfetschy、Pisinger , *Knapsack Problems* (Springer , 2004) ; Martello & Toth , *Knapsack Problems: Algorithms and Computer Implementations* (Wiley , 1990) : 背包问题的专著，含上界、分支定界、动态规划与近似算法。
- Dantzig , “Discrete-variable extremum problems” (Operations Research , 1957) : 背包 LP 上界。Balas , “Facets of the knapsack polytope” (1975) ; Wolsey , “Faces for a linear inequality in 0–1 variables” (1975) : 覆盖不等式。
- Ross & Soland , “A branch and bound algorithm for the generalized assignment problem” (Mathematical Programming , 1975) ; Shmoys & Tardos , “An

approximation algorithm for the generalized assignment problem” (1993) : GAP 的精确与近似算法。

- Dósa , “The tight bound of First Fit Decreasing bin-packing algorithm is $FFD(I) \leq 11/9 OPT(I) + 6/9$ ” (2007) ; Gilmore & Gomory , “A linear programming approach to the cutting-stock problem” (1961) : 装箱与列生成的起点。

本章 Lab

代码包 `ch07-capacitated-lab.zip` 随本章一起发放。依赖是 `numpy`、`scipy`、`pytest` ; 判分用暴力枚举、动态规划和 `linprog` , 不依赖你的代码。

练习	内容	对应
ex1	背包 : Dantzig 上界 (支持固定变量) 与动态规划 ; 先填 4 个预测	7.1
ex2	背包的分支定界 : 按规定的分支和剪枝规则 , 节点数要与判分数据一致 (页面的例子是 15)	7.2
ex3	广义指派 : LP 松弛 (可带割) 与回溯枚举	7.3
ex4	覆盖割 : 分离极小覆盖 , 根节点割平面循环 ; 验证割对每个整数可行解有效	7.4
ex5	审 5 个 AI 写的 GAP 求解函数 : 不看源码 , 只靠运行结果归类问题	7.3
挑战	松弛载重的拉格朗日下界 : 次梯度上升 , 验证它追得近 LP 值 , 追不到整数最优	7.4

```
pip install -r requirements.txt
pytest -q # 全部练习
pytest tests/test_ex2.py -q # 单个练习

# 确认判分本身没问题 : 参考答案应当全部通过
LAB_TARGET=solutions pytest -q
```

练习 1 要求先在 `PREDICTION` 里写下预测再运行 ; 跑完测试后 , 终端最后会把你的预测和实际结果并排列出来。

下一章

CHAPTER 08

旅行商与车辆路径入门

本章的覆盖割是“从 LP 解里读出割”的第一个例子。下一章遇到第二个：子回路消除。一辆车要走遍若干个点再回来（旅行商问题），LP 会给出几个互不相连的小圈，割就是“这些点必须有一条路通向外面”。再把本章的载重和下一章的路线拼在一起，就是车辆路径问题的模型骨架。

7.8 自测

8 道题，覆盖本章的全部要点。每题选完立即给出解析。

Q1 0-1 背包的 LP 松弛，最优解里最多有几个变量取分数？为什么？

最多 2 个，因为有上界约束

任意多个

0 个，因为背包的矩阵全单模

最多 1 个：只有一行约束，顶点上至多一个变量落在 0 和 1 之间

正确。 按价值/重量装，装到装不下的那个装一部分，其余不是 0 就是 1。矩阵只有一行，系数 w_j 是任意整数，不是全单模，所以 LP 的最优值可以高于整数最优。

Q2 Dantzig 上界是 106.86，整数最优是 104。关于“上界向下取整”，哪个说法对？

向下取整得 106，就是最优

向下取整仍是上界，但不一定等于最优；这里 106 大于 104

向下取整会低于最优

价值是整数时上界一定是整数

正确。 整数解也满足 LP 的约束，所以最优不超过上界，取整之后（价值是整数）仍是上界。它可以比最优大，这时还要分支或者别的手段来收紧。

Q3 背包的分支定界里，节点上界向下取整不超过手上最好的整数解，为什么可以剪掉这个节点？

因为它的 LP 是分数

因为这个节点已经没有可行解了

价值是整数，这个节点里任何整数解的价值都不超过上界的整数部分，不可能比手上的更好

因为它是深度最深的节点

正确。 这一支里的整数解都满足这一支的 LP 约束，价值不超过 LP 值；价值是整数，所以不超过它的整数部分。不比手上的更好，就不用再看。

Q4 为什么广义指派 (GAP) 的 LP 不再自带整数解，而第 5 章的指派可以？

因为 GAP 没有目标函数

因为求解器变慢了

因为变量更多了

第 5 章的约束矩阵每列恰有两个 1 (二部图)，全单模；载重约束的系数是 w_j ，破坏了这个结构

正确。 全单模的证明依赖“每列一个任务行的 1 加一个车行的 1”。载重行的系数是任意的托盘数，这个结构没有了，LP 的顶点可以是分数点，表现为一个任务被劈给几辆车。

Q5 图 7-3 里把每辆车的载重加 2 以后，LP 值等于整数最优 (212)。原因是？

三辆车分到的托盘数 (每个任务拿走最便宜的车) 都在载重之内，载重约束不起作用，问题退化成“每个任务选最便宜的车”

求解器的容差变大了

任务变少了

载重约束变成了等式

正确。 载重不起作用时，GAP 就是没有容量的指派，整数性回来。载重紧到会起作用，LP 才会把一个任务劈开。

Q6 覆盖割 $\sum_{j \in S} x_{ij} \leq |S| - 1$ 为什么是“有效”的，为什么又是“有用”的？

它对所有整数可行解都成立（全装进去就超载了），但会删掉当前 LP 的分数解

它对所有分数解都成立

它会删掉一些整数最优解，但更快

它只在 x 全为 0 时成立

正确。 S 的总重量超过载重，整数解不可能把 S 里的任务全装在车 i 上，所以整数可行解都满足这条不等式，没有整数解被删掉。LP 里 x 的和超过 $|S| - 1$ 的分数解被删掉，这就是割的作用。

Q7 松弛载重约束得到的拉格朗日下界，最好能达到多少？

0

整数最优

LP 值：松弛后的子问题（每个任务选加价后最便宜的车）有整数性，所以最优 λ 给出的下界等于 LP 值

无限大

正确。 子问题的 LP 已经是整数的（每个任务独立地选车），拉格朗日对偶的最优值等于 LP 值。覆盖割能超过 LP 值，是因为它利用了“装不下”这个整数性质。

Q8 装箱里，FFD 用了 4 个车次，下界是 $\lceil 54/18 \rceil = 3$ 。关于最优车次，正确的结论是？

最优一定是 3

最优可能大于 4

最优一定是 4

最优在 3 和 4 之间，要用别的办法（比如回溯）确定；这个例子里最优是 3

正确。 FFD 的车次数是最优的上界，下界给出最优的下限，所以最优在 3 和 4 之间。要确定就要搜索。这个例子里最优是 3，每个车次正好装满，FFD 多开了一个车次。

8 / 8 本章掌握得不错，可以去做 Lab 了。

数据说明：本章的全部数字由页面内置的背包上界、动态规划、分支定界、单纯形法、覆盖割循环和回溯实时计算，并与暴力枚举、SciPy 的 `linprog` 和 HiGHS 的 MIP 逐项对拍（环境：SciPy 1.17.1 · HiGHS 1.12.0）：随机背包的 LP 上界与最优、随机广义指派的 LP 值与整数最优、随机装箱的最优车次与 FFD，都与 Lab 的判分数据同源。